

DEVOPS MASTER COURSE • VOLUME 1 • LINUX

DEVOPS MASTER COURSE

VOLUME 1

Linux Fundamentals & Administration

Beginner to Junior DevOps Engineer

- ✓ Theory from zero
- ✓ Hands-on labs
- ✓ How companies use Linux
- ✓ Troubleshooting playbooks
- ✓ Projects and interview questions
- ✓ Completion tracking

Edition: July 2026

How to Use This Book

This volume is designed for continuous study. Complete each lesson in order, run every command in a safe Linux lab, finish the exercises, and mark the unit Completed only when you can explain the topic without reading the answer.

Recommended lab: Use Ubuntu Server in VirtualBox, VMware, WSL2, or a cloud VM. Never practise destructive commands on a production machine.

Completion Rules

- ☐ Read the concept and explain it in your own words.
- ☐ Type every command manually.
- ☐ Predict the output before running commands.
- ☐ Complete the lab and troubleshooting task.
- ☐ Answer at least 80% of the checkpoint questions.
- ☐ Mark the unit Completed and record the date.

Volume Outcomes

After completing this volume you should be able to operate a Linux server, manage users and permissions, control processes and services, inspect logs, troubleshoot network and storage issues, use SSH safely, and explain how Linux is used in a DevOps team.

Master Progress Dashboard

Unit	Topic	Status	Completion date
1	Linux and DevOps foundations	☐ Not Started	_____
2	Lab setup and terminal basics	☐ Not Started	_____
3	Filesystem and navigation	☐ Not Started	_____
4	Files, directories, text and search	☐ Not Started	_____
5	Users, groups and identity	☐ Not Started	_____
6	Permissions, ownership and sudo	☐ Not Started	_____
7	Processes and job control	☐ Not Started	_____
8	systemd and service management	☐ Not Started	_____
9	Logs and debugging	☐ Not Started	_____
10	Networking fundamentals	☐ Not Started	_____
11	Packages and software management	☐ Not Started	_____
12	Storage, disks and	☐ Not Started	_____

Continuous Learning Edition • Mark each unit Completed before advancing

DEVOPS MASTER COURSE • VOLUME 1 • LINUX

	filesystems		
13	SSH and remote administration	☐ Not Started	_____
14	Security and hardening basics	☐ Not Started	_____
15	Performance and resource troubleshooting	☐ Not Started	_____
16	Scheduling with cron and at	☐ Not Started	_____
17	Archives, compression and transfers	☐ Not Started	_____
18	Company workflow and incident handling	☐ Not Started	_____
19	Capstone projects	☐ Not Started	_____
20	Interview preparation and final exam	☐ Not Started	_____

Unit 1 - Linux and DevOps Foundations

Learning objectives

- Explain the role of Linux in DevOps
- Describe the kernel, shell and user space
- Identify common Linux distributions

What is Linux?

Linux is an operating-system kernel. A complete Linux operating system combines the Linux kernel with system tools, libraries, package managers and applications. Ubuntu, Debian, Red Hat Enterprise Linux, Rocky Linux and Amazon Linux are distributions built around the kernel.

Most servers, containers, CI runners, Kubernetes nodes and cloud virtual machines use Linux because it is stable, automatable, scriptable and widely supported.

Kernel, shell and commands

The kernel manages CPU, memory, disks, devices, networking and processes. The shell accepts commands and starts programs. Bash is a common shell used in DevOps. Commands such as `ls`, `cp` and `systemctl` are programs executed by the shell.

```
echo $SHELL
uname -a
cat /etc/os-release
```

Linux architecture

A simplified path is: user -> terminal -> shell -> system calls -> kernel -> hardware. Applications usually do not access hardware directly; they ask the kernel for resources.

Distributions and package families

Debian-family systems commonly use `apt` and `.deb` packages. Red Hat-family systems use `dnf/yum` and `.rpm` packages. Commands and paths are largely similar, but package names and security tools can differ.

How companies use this

DevOps teams standardise server images, package versions and configuration so environments remain predictable. Linux is commonly the base operating system for web servers, databases, CI agents and Kubernetes worker nodes.

Hands-on lab

1. Open a terminal.
2. Run `uname -a` and identify the kernel version.
3. Run `cat /etc/os-release` and identify the distribution.

4. Run `echo $SHELL` and identify your shell.

5. Write a five-line explanation of why Linux is central to DevOps.

Checkpoint

- ☐ I can explain kernel versus shell.
- ☐ I can name three Linux distributions.
- ☐ I can explain why cloud servers commonly use Linux.

Interview questions

Q: Is Linux an operating system or a kernel?

A: Technically Linux is the kernel; distributions combine it with user-space tools to form a complete operating system.

Q: Why is Linux popular in DevOps?

A: It is stable, automatable, secure, cloud-friendly and supported by most infrastructure tools.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 2 - Lab Setup and Terminal Basics

Learning objectives

- ☐ Create a safe Linux learning environment
- ☐ Use terminal shortcuts and command history
- ☐ Get help for unfamiliar commands

Choosing a lab

Suitable options are an Ubuntu virtual machine, WSL2 on Windows, a spare machine, or a low-cost cloud VM. A virtual machine is ideal because snapshots let you recover from mistakes.

Prompt and command structure

A command normally has a command name, options and arguments. In `ls -lah /var/log`, `ls` is the command, `-lah` contains options and `/var/log` is the argument. Linux is case-sensitive.

```
ls -lah /var/log
pwd
whoami
```

Help systems

Use `man` for manuals, `--help` for quick usage, and `apropos` to find commands by topic.

```
man ls
ls --help
apropos network
```

History and shortcuts

Use history to review commands. `Ctrl+C` interrupts a foreground command. `Ctrl+L` clears the screen. `Tab` completes paths and commands. Up Arrow recalls previous commands.

```
history
!!
!25
```

Hands-on lab

1. Create or start your Linux lab.
2. Take a VM snapshot named `clean-linux-lab`.
3. Run `pwd`, `whoami`, `hostname` and `date`.
4. Open `man ls`, search for reverse sorting using `/reverse`, then quit with `q`.
5. Use `Tab` completion to enter `/var/log`.

Checkpoint

- ☐ I can distinguish command, option and argument.

☐ I can use man and --help.

☐ I know what Ctrl+C, Ctrl+L and Tab do.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 3 - Filesystem and Navigation

Learning objectives

- ☐ Understand Linux directory structure
- ☐ Navigate using absolute and relative paths
- ☐ Inspect files and directories safely

Filesystem hierarchy

The filesystem begins at /. Important paths include /home for user data, /etc for configuration, /var for changing data and logs, /usr for applications and libraries, /tmp for temporary files, /opt for optional software, /srv for service data and /root for the root user.

Absolute and relative paths

An absolute path begins with /. A relative path begins from the current directory. The symbols . and .. mean current directory and parent directory. The tilde ~ refers to the current user home directory.

```
cd /var/log
cd ..
cd ~
pwd
```

Listing content

ls lists directory entries. -l shows details, -a includes hidden entries, -h makes sizes readable, and -t sorts by modification time.

```
ls -lah
ls -lt /var/log
```

Hands-on lab

1. Create ~/devops-lab/unit3.
2. Inside it create directories app, logs and backup.
3. Navigate using both absolute and relative paths.
4. Create a hidden file named .env.
5. Run ls -lah and explain every column in the output.

Checkpoint

- ☐ I know what /, ~, . and .. mean.
- ☐ I can identify /etc, /var/log and /home.
- ☐ I can use absolute and relative paths.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint

without notes.

Unit 4 - Files, Directories, Text and Search

Learning objectives

- ☐ Create, copy, move and remove files
- ☐ Read and search text files
- ☐ Use pipes and redirection

File operations

touch creates an empty file or updates timestamps. mkdir creates directories. cp copies, mv moves or renames, and rm removes. rm -rf is destructive and must be used with extreme care.

```
touch app.log
mkdir -p project/{config,logs,backup}
cp app.log backup/
mv app.log application.log
```

Reading files

cat is suitable for short files. less is better for large files. head and tail show the beginning or end. tail -f follows appended log lines in real time.

```
cat /etc/hostname
less /var/log/syslog
head -n 10 file.txt
tail -f app.log
```

Searching

grep searches text. find searches paths and metadata. Use grep -i for case-insensitive matching, -r for recursive search and -n for line numbers.

```
grep -in "error" app.log
find /var/log -type f -name "*.log" 2>/dev/null
```

Pipes and redirection

A pipe sends one command output into another command. > overwrites a file, >> appends, and 2> redirects errors.

```
ps aux | grep nginx
echo "start" > app.log
echo "next" >> app.log
find /root 2> errors.txt
```

Hands-on lab

1. Create a ten-line log file containing INFO, WARNING and ERROR messages.
2. Use grep to show only ERROR lines with line numbers.
3. Use tail to show the final three lines.
4. Redirect the result into errors-only.log.

5. Use find to locate every .log file inside your lab.

Checkpoint

- ☐ I understand > versus >>.
- ☐ I can explain a pipe.
- ☐ I can use grep and find for troubleshooting.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 5 - Users, Groups and Identity

Learning objectives

- ☐ Inspect users and groups
- ☐ Create and manage accounts
- ☐ Understand UID, GID and login identity

Identity model

Each user has a UID and primary GID. Groups allow shared access. Account information is stored in `/etc/passwd`; password hashes are protected in `/etc/shadow`; group information is stored in `/etc/group`.

```
id
whoami
groups
getent passwd $USER
```

User management

Administrative commands include `useradd` or `adduser`, `passwd`, `usermod` and `userdel`. On Ubuntu, `adduser` provides an interactive workflow.

```
sudo adduser devopsstudent
sudo usermod -aG sudo devopsstudent
sudo passwd devopsstudent
```

Switching identity

`su` starts a shell as another user. `sudo` runs a command with delegated privileges. `sudo -i` starts a root login shell when permitted.

```
su - devopsstudent
sudo whoami
sudo -i
```

Hands-on lab

1. Create a user named `labuser`.
2. Create a group named `devteam`.
3. Add `labuser` to `devteam` without removing existing group membership.
4. Verify membership with `id labuser`.
5. Lock and unlock the account, then remove it after the lab.

Checkpoint

- ☐ I can explain UID and GID.
- ☐ I know why `usermod -aG` includes `-a`.

☐ I understand su versus sudo.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 6 - Permissions, Ownership and sudo

Learning objectives

- ☐ Read symbolic and numeric permissions
- ☐ Change mode and ownership
- ☐ Apply least privilege

Permission structure

Permissions are read, write and execute for owner, group and others. For files, execute means runnable. For directories, execute means traversal. Numeric values are read=4, write=2 and execute=1.

```
ls -l
chmod 750 deploy.sh
chmod u+x deploy.sh
```

Ownership

chown changes owner and optionally group. chgrp changes group. Recursive changes should be reviewed carefully.

```
sudo chown appuser:devteam application.conf
sudo chgrp devteam application.conf
```

Special permissions

setuid, setgid and sticky bit are special modes. The sticky bit on shared directories such as /tmp prevents users from deleting files owned by others.

```
ls -ld /tmp
chmod 1777 shared-dir
```

sudo policy

sudo permissions are configured in /etc/sudoers and /etc/sudoers.d. Always edit using visudo to validate syntax. Grant only commands required for the role.

```
sudo visudo
```

Hands-on lab

1. Create deploy.sh and set permission 750.
2. Create a shared directory owned by group devteam with mode 2770.
3. Explain why 777 is usually unsafe.
4. Use stat to inspect file ownership and numeric mode.
5. Create a safe sudoers example that permits restarting nginx only.

Checkpoint

- ☐ I can translate `rwxr-x---` into 750.
- ☐ I understand directory execute permission.
- ☐ I can explain least privilege.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 7 - Processes and Job Control

Learning objectives

- ☐ Inspect running processes
- ☐ Send signals safely
- ☐ Manage foreground and background jobs

Processes

Every running program has a PID. ps provides snapshots. top and htop provide interactive monitoring. Important metrics include CPU, memory, state, user and elapsed time.

```
ps aux
ps -ef
top
```

Signals

kill sends a signal to a PID. SIGTERM asks a process to exit gracefully. SIGKILL forces termination and should be a last resort. killall and pkill target names or patterns.

```
kill 2451
kill -15 2451
kill -9 2451
pkill -f "python app.py"
```

Jobs

Appending & starts a command in the background. jobs lists shell jobs. fg and bg move jobs between foreground and background. nohup helps a process continue after logout, although managed services are preferred in production.

```
sleep 300 &
jobs
fg %1
nohup ./app.sh > app.out 2>&1 &
```

Hands-on lab

1. Start sleep 300 in the background.
2. Find its PID using ps or pgrep.
3. Send SIGTERM and confirm it exits.
4. Start another sleep process and practise Ctrl+Z, bg and fg.
5. Explain when SIGKILL is justified.

Checkpoint

- ☐ I can find a PID.

☐ I know SIGTERM versus SIGKILL.

☐ I can manage jobs.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 8 - systemd and Service Management

Learning objectives

- ☐ Understand units and service lifecycle
- ☐ Start, stop, restart and enable services
- ☐ Create a basic custom service

systemd

systemd is the init and service manager on many modern Linux distributions. Units describe services, timers, mounts, sockets and other resources.

Core commands

start changes the current state; enable configures startup at boot. reload asks a service to reread configuration when supported. status shows state and recent logs.

```
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl restart nginx
sudo systemctl reload nginx
systemctl status nginx
```

Custom unit

A service file normally belongs in /etc/systemd/system. After editing units, run daemon-reload.

```
[Unit]
Description=Demo Application
After=network.target

[Service]
ExecStart=/usr/local/bin/demo-app
Restart=on-failure
User=demo

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
```

Hands-on lab

1. Install nginx in your lab.
2. Start it and confirm active status.
3. Enable it and verify is-enabled.
4. Stop it, test the endpoint, then start it again.
5. Create a harmless custom oneshot service that writes the date to /tmp/service-test.txt.

Checkpoint

- ☐ I understand start versus enable.
- ☐ I can use status and is-enabled.
- ☐ I know why daemon-reload is required.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 9 - Logs and Debugging

Learning objectives

- ☐ Locate system and application logs
- ☐ Use journalctl effectively
- ☐ Follow a structured debugging process

Log sources

Traditional logs often live under /var/log. systemd collects journal events accessed through journalctl. Applications may log to files, stdout/stderr, syslog or a central platform.

```
ls -lah /var/log
journalctl -n 50
```

Service logs

Filter by unit, time, priority or boot. Use -f to follow.

```
journalctl -u nginx
journalctl -u nginx -f
journalctl --since "30 minutes ago"
journalctl -p err -b
```

Debugging method

Use the sequence: define symptom, reproduce, check service state, inspect logs, confirm configuration, confirm dependencies, test locally, test network path, change one thing, verify and document.

Log filtering

Combine grep, awk, sort and uniq to summarise repeated events.

```
grep -i "error" app.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head
```

Hands-on lab

1. Break the nginx configuration deliberately with a safe typo after making a backup.
2. Run nginx -t and record the error.
3. Restart nginx and inspect systemctl status plus journalctl -u nginx.
4. Restore the configuration and verify recovery.
5. Write a mini incident note containing symptom, cause, fix and prevention.

Checkpoint

- ☐ I can use journalctl -u and -f.
- ☐ I check logs before guessing.

☐ I can describe a repeatable troubleshooting method.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 10 - Networking Fundamentals

Learning objectives

- ☐ Understand IP, ports, protocols and DNS
- ☐ Inspect interfaces, routes and listening services
- ☐ Troubleshoot connectivity layer by layer

Core concepts

An IP address identifies a network interface. A port identifies an application endpoint. TCP is connection-oriented; UDP is connectionless. DNS translates names to addresses. Common ports include 22 SSH, 53 DNS, 80 HTTP and 443 HTTPS.

Interface and route commands

ip replaces older ifconfig and route commands on modern systems.

```
ip address
ip link
ip route
```

Connectivity tests

ping tests ICMP reachability when allowed. curl tests application endpoints. ss shows sockets. dig or nslookup tests DNS. traceroute shows the network path.

```
ping -c 4 8.8.8.8
curl -I http://localhost
ss -tulnp
dig example.com
traceroute example.com
```

Troubleshooting order

Check local process, listening port, local curl, firewall, interface, route, DNS, remote reachability, load balancer and application logs. A successful ping does not prove the application is healthy.

Hands-on lab

1. Run ip a and identify your active interface.
2. Run ip route and identify the default gateway.
3. Start nginx and confirm port 80 with ss -tulnp.
4. Run curl -I localhost and interpret the HTTP status.
5. Use dig to resolve a domain and identify the returned IP address.

Checkpoint

- ☐ I can explain IP versus port.

☐ I can use ss and curl.

☐ I can separate DNS, transport and application failures.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 11 - Packages and Software Management

Learning objectives

- ☐ Use apt and dnf/yum safely
- ☐ Understand repositories and package metadata
- ☐ Verify installed software

Package managers

Package managers resolve dependencies, verify packages and track installed files. Ubuntu/Debian commonly use apt; RHEL-family systems use dnf.

```
sudo apt update
sudo apt install nginx
apt list --installed
sudo apt remove nginx
```

Red Hat family

Equivalent commands use dnf.

```
sudo dnf check-update
sudo dnf install nginx
dnf list installed
```

Operational safety

In companies, package changes are tested, reviewed and automated. Avoid unplanned upgrades on production servers. Record versions and maintain rollback plans.

```
nginx -v
dpkg -l | grep nginx
rpm -qa | grep nginx
```

Hands-on lab

1. Install tree and jq.
2. Verify their versions.
3. Identify which package owns a binary.
4. Remove one package and verify removal.
5. Explain apt update versus apt upgrade.

Checkpoint

- ☐ I can install and remove packages.
- ☐ I understand repository metadata.
- ☐ I avoid uncontrolled production upgrades.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 12 - Storage, Disks and Filesystems

Learning objectives

- ☐ Inspect disks and filesystem usage
- ☐ Find large files and directories
- ☐ Understand mounts, inodes and LVM basics

Disk views

lsblk shows block devices. df reports filesystem usage. du measures directory/file usage. A filesystem can run out of blocks or inodes.

```
lsblk -f
df -h
df -i
du -xhd1 /var | sort -h
```

Mounting

A filesystem is attached to the directory tree at a mount point. /etc/fstab describes persistent mounts. Incorrect fstab entries can affect boot, so test with mount -a.

```
findmnt
cat /etc/fstab
sudo mount -a
```

LVM overview

Logical Volume Manager separates physical storage from logical volumes. It supports flexible resizing and snapshots, but operations must be planned and backed up.

```
sudo pvs
sudo vgs
sudo lvs
```

Hands-on lab

1. Run lsblk -f and map devices to mount points.
2. Run df -h and identify the fullest filesystem.
3. Use du to find the largest directory inside your lab.
4. Create many small files and observe inode usage with df -i.
5. Explain why deleting an open log may not immediately free space.

Checkpoint

- ☐ I know df versus du.
- ☐ I can investigate a full disk.

☐ I understand block usage versus inode usage.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 13 - SSH and Remote Administration

Learning objectives

- Connect securely to remote Linux systems
- Use key-based authentication
- Transfer files and troubleshoot SSH

SSH basics

SSH provides encrypted remote login and command execution. The client connects to sshd, usually on TCP port 22.

```
ssh user@server-ip
ssh -p 2222 user@server-ip
```

Keys

A key pair includes a private key kept secret and a public key installed on the server. Protect private keys with correct permissions and preferably a passphrase.

```
ssh-keygen -t ed25519
ssh-copy-id user@server-ip
chmod 600 ~/.ssh/id_ed25519
```

Transfers

scp copies files. rsync efficiently synchronises content and can preserve attributes.

```
scp app.tar.gz user@server:/tmp/
rsync -avz project/ user@server:/srv/project/
```

Troubleshooting

Use ssh -v for verbose diagnostics. Check network reachability, port 22, sshd status, firewall, user account, key permissions and server logs.

```
ssh -v user@server
sudo systemctl status ssh
sudo journalctl -u ssh
```

Hands-on lab

1. Create an SSH key pair in your lab.
2. Inspect public and private key permissions.
3. Connect to a second VM or localhost SSH server using a key.
4. Transfer a test file using scp.
5. Use ssh -v and identify authentication stages.

Checkpoint

- ☐ I understand public versus private keys.
- ☐ I can use scp and rsync.
- ☐ I can troubleshoot SSH authentication.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 14 - Security and Hardening Basics

Learning objectives

- ☐ Apply least privilege and secure defaults
- ☐ Use firewall and update practices
- ☐ Recognise common Linux security controls

Security principles

Use least privilege, defence in depth, secure configuration, timely patching, strong authentication, logging and tested backups. Disable unused services and ports.

Firewall

Ubuntu commonly uses ufw as a front end. RHEL commonly uses firewalld. Always preserve SSH access before enabling remote firewall rules.

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw enable
sudo ufw status verbose
```

SSH hardening

Prefer keys, restrict root login, remove unused accounts and review authentication logs. Test configuration before reloading sshd.

```
sudo sshd -t
sudo journalctl -u ssh --since today
```

Mandatory access control

SELinux and AppArmor apply policy beyond standard Unix permissions. Do not disable them as a first troubleshooting step; inspect denials and correct policy or context.

```
getenforce
sestatus
sudo aa-status
```

Hands-on lab

1. List listening ports and identify unnecessary services.
2. Enable a firewall in the lab while preserving SSH access.
3. Review failed login events.
4. Check for available security updates.
5. Write a ten-point Linux hardening checklist.

Checkpoint

- ☐ I understand least privilege.
- ☐ I can describe firewall purpose.
- ☐ I know why disabling SELinux/AppArmor is poor practice.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 15 - Performance and Resource Troubleshooting

Learning objectives

- ☐ Interpret CPU, memory, load and I/O signals
- ☐ Find resource-heavy processes
- ☐ Use a structured performance workflow

CPU and load

Load average represents runnable or uninterruptible work over 1, 5 and 15 minutes. Compare load with CPU count. High load may come from CPU or I/O waits.

```
uptime
nproc
top
```

Memory

free shows memory totals and available memory. Linux uses free memory for cache; low free memory alone is not necessarily a problem. Watch available memory, swap and OOM events.

```
free -h
vmstat 1
journalctl -k | grep -i oom
```

I/O and files

iostat, pidstat and lsof help investigate disks and open files. These may require sysstat or lsof packages.

```
iostat -xz 1
pidstat 1
lsof +L1
```

Workflow

Measure before changing. Identify the constrained resource, isolate the process, correlate with logs and deployments, reduce impact, fix root cause and monitor after recovery.

Hands-on lab

1. Record uptime, CPU count and load average.
2. Identify the top three CPU and memory processes.
3. Run a temporary CPU load in the lab and observe top.
4. Inspect swap usage.
5. Use lsof +L1 to check for deleted-but-open files.

Checkpoint

- ☐ I can explain load average.

☐ I understand available memory.

☐ I measure before tuning.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 16 - Scheduling with cron and at

Learning objectives

- ☐ Schedule recurring and one-time jobs
- ☐ Manage cron environment and logs
- ☐ Prevent duplicate or unsafe jobs

Cron syntax

Cron fields are minute, hour, day of month, month and day of week. Commands run with a limited environment, so use absolute paths and redirect output.

```
crontab -e
# m h dom mon dow command
0 2 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
```

One-time jobs

at schedules a one-time command when the at service is available.

```
echo "/usr/local/bin/report.sh" | at 18:00
```

Reliability

Scripts should use locking, clear exit codes, logging and alerting. systemd timers are often preferred for richer control and observability.

Hands-on lab

1. Create a script that appends date to /tmp/cron-test.log.
2. Schedule it every minute temporarily.
3. Verify execution and locate cron logs.
4. Remove the test schedule.
5. Explain how you would prevent overlapping backup jobs.

Checkpoint

- ☐ I can read cron fields.
- ☐ I use absolute paths.
- ☐ I know cron has a limited environment.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 17 - Archives, Compression and Transfers

Learning objectives

- ☐ Create and extract archives
- ☐ Choose common compression formats
- ☐ Verify backups and transfers

tar and compression

tar groups files into an archive. gzip and xz compress data. Common extensions include .tar.gz and .tar.xz.

```
tar -czf project.tar.gz project/  
tar -tzf project.tar.gz  
tar -xzf project.tar.gz
```

Integrity

A backup is not complete until restoration is tested. Checksums detect corruption or transfer changes.

```
sha256sum project.tar.gz  
sha256sum -c project.tar.gz.sha256
```

Transfers

Use rsync for repeatable synchronisation and scp for simple copies. In production, define encryption, retention and restore tests.

Hands-on lab

1. Archive your lab directory.
2. List archive contents without extraction.
3. Create a SHA-256 checksum.
4. Extract into a new directory and compare files.
5. Document a backup retention and restore-test plan.

Checkpoint

- ☐ I can create and extract tar.gz.
- ☐ I can verify checksums.
- ☐ I understand that backup success requires restore testing.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint without notes.

Unit 18 - Company Workflow and Incident Handling

Learning objectives

- ☐ Understand daily DevOps Linux work
- ☐ Follow change and incident processes
- ☐ Write operational documentation

Daily workflow

A DevOps engineer may review alerts, investigate incidents, support deployments, manage configuration, improve automation, patch systems, review capacity and document changes. Work is usually tracked in tickets and repositories.

Change management

A production change should have scope, owner, risk, test evidence, rollout steps, rollback steps, monitoring and approval. Emergency changes still require documentation after stabilisation.

Incident lifecycle

Detect, acknowledge, assess severity, stabilise service, communicate, investigate, recover, verify, document timeline, perform root-cause analysis and track corrective actions. Avoid blame; focus on system improvements.

Runbooks and postmortems

A runbook describes repeatable operational actions. A postmortem documents impact, timeline, root cause, contributing factors, resolution and prevention.

Hands-on lab

1. Write a change plan to upgrade nginx in production.
2. Include validation and rollback steps.
3. Simulate an nginx outage and write an incident timeline.
4. Create a one-page nginx recovery runbook.
5. Write three preventive actions after the incident.

Checkpoint

- ☐ I can distinguish incident, problem and change.
- ☐ I can write rollback steps.
- ☐ I understand blameless postmortems.

Completion gate: Mark this unit Completed only after the lab works and you can answer the checkpoint

without notes.

Unit 19 - Capstone Projects

Complete all three projects. Use a Git repository for notes, commands, configuration and evidence. Do not include passwords or private keys.

Project A - Secure Web Server

Provision an Ubuntu VM, create a non-root admin user, configure SSH keys, install nginx, enable a firewall, publish a test page, configure service startup, and document validation plus rollback.

- ☐ Architecture or setup diagram
- ☐ Commands and configuration
- ☐ Test evidence
- ☐ Security checks
- ☐ Troubleshooting notes
- ☐ Rollback or recovery process
- ☐ Final lessons learned

Project B - Linux Health Report

Create a command-line health report that records hostname, date, uptime, CPU count, load, memory, filesystem usage, top processes, listening ports and failed services. Shell scripting implementation will be expanded in Volume 2.

- ☐ Architecture or setup diagram
- ☐ Commands and configuration
- ☐ Test evidence
- ☐ Security checks
- ☐ Troubleshooting notes
- ☐ Rollback or recovery process
- ☐ Final lessons learned

Project C - Incident Simulation

Break a safe lab service, detect the outage, collect status and logs, identify root cause, restore service, verify externally, and write a postmortem with corrective actions.

- ☐ Architecture or setup diagram
- ☐ Commands and configuration

- ☐ Test evidence
- ☐ Security checks
- ☐ Troubleshooting notes
- ☐ Rollback or recovery process
- ☐ Final lessons learned

Capstone acceptance criteria

- ☐ All services survive reboot.
- ☐ Only required ports are listening.
- ☐ No private key or password is stored in project files.
- ☐ Another learner can follow the documentation.
- ☐ Failure and recovery were tested, not merely described.

Unit 20 - Interview Preparation and Final Exam

Interview Question Bank

Q: What happens after you type a command in Bash?

A: Bash parses the command, performs expansions and redirections, locates the executable or built-in, starts it, and the program uses system calls to request kernel services.

Q: What is the difference between a process and a service?

A: A process is a running program. A service is a long-running function normally managed by a service manager such as systemd and may include restart and boot policies.

Q: What is the difference between hard and symbolic links?

A: A hard link is another directory entry for the same inode and normally cannot cross filesystems. A symbolic link stores a path and can cross filesystems but can become broken.

Q: Why can df and du show different values?

A: Deleted but still-open files, filesystem metadata, reserved blocks, mount boundaries and sparse files can cause differences.

Q: What does chmod 750 mean?

A: Owner has rwx, group has r-x, and others have no permissions.

Q: How do you investigate a full disk?

A: Use df -h and df -i, identify the filesystem, use du within the same filesystem, inspect logs and deleted-open files with lsof +L1, then remediate safely.

Q: How do you investigate an inaccessible web application?

A: Check process/service, logs, listening port, local curl, configuration, dependencies, firewall, route/DNS/load balancer and remote curl.

Q: Why use SIGTERM before SIGKILL?

A: SIGTERM allows graceful cleanup; SIGKILL cannot be handled and may cause data loss or inconsistent state.

Q: What is load average?

A: The average number of runnable and uninterruptible tasks over 1, 5 and 15 minutes; interpret it relative to CPU count and I/O behaviour.

Q: What is the difference between systemctl start and enable?

A: start changes the current running state; enable configures the unit to start automatically according to boot targets.

Q: How do you find which process owns a port?

A: Use `ss -ltnp` or `ss -tulnp`, optionally filtering the port.

Q: What is an inode?

A: Filesystem metadata object representing a file, excluding its filename; it stores ownership, mode, timestamps and block pointers.

Q: How does sudo differ from su?

A: sudo runs authorised commands under another identity using policy and auditing; su switches to another user shell and commonly requires that account password or root access.

Q: What should a production change include?

A: Scope, risk, owner, test evidence, implementation, validation, monitoring, rollback and communication.

Q: Why should logs be centralised?

A: Centralisation supports searching across hosts, retention, access control, alerting and investigation even if a server is lost.

Scenario Questions

- ☐ A server is reachable by ping but HTTPS fails. Explain your diagnostic order.
- ☐ CPU is normal but load average is high. What do you inspect?
- ☐ A filesystem is 100% full, but du totals are much lower than df. What might be happening?
- ☐ An SSH key works for one user but fails for another. What permissions and logs do you inspect?
- ☐ A service starts manually but fails after reboot. What systemd properties do you check?
- ☐ A cron job works in your terminal but fails through cron. Why?
- ☐ After deployment, nginx returns 502 Bad Gateway. What does that suggest and how do you verify?
- ☐ Users report intermittent slowness. Which evidence do you collect before restarting anything?
- ☐ A teammate proposes `chmod 777` to fix a permission problem. How do you respond?
- ☐ A package upgrade breaks production. What should have existed before the change?

Practical Final Exam

- ☐ Create a Linux user and restricted shared group.
- ☐ Install and configure nginx.

- ☐ Serve a page from a controlled directory.
- ☐ Configure permissions without using 777.
- ☐ Enable SSH key authentication.
- ☐ Allow only SSH and HTTP through the firewall.
- ☐ Prove nginx listens on port 80.
- ☐ Collect service logs and system resource evidence.
- ☐ Schedule a health report.
- ☐ Create a backup archive and verify its checksum.
- ☐ Simulate one failure and recover.
- ☐ Write change, validation, rollback and incident notes.

Passing standard

Pass when you complete at least 10 of 12 practical tasks without copying commands blindly, score at least 80% in oral questioning, and can explain your troubleshooting logic. After passing, proceed to Volume 2: Bash Shell Scripting and Automation.

Command Reference

Area	Essential commands
Identity	whoami, id, groups, who, last
Filesystem	pwd, ls -lah, cd, find, stat, file
Files	touch, mkdir -p, cp -a, mv, rm, ln -s
Text	cat, less, head, tail -f, grep, awk, sed, sort, uniq, cut
Processes	ps aux, top, htop, pgrep, kill, pkill, jobs, fg, bg
Services	systemctl status, systemctl restart, systemctl enable, journalctl -u
Network	ip a, ip route, ss -tulnp, ping, curl, dig, traceroute
Storage	lsblk -f, df -h, df -i, du -xhd1, findmnt, lsof +L1
Packages	apt, dnf, dpkg, rpm
Remote	ssh, scp, rsync, ssh-keygen
Archives	tar, gzip, xz, sha256sum

Final Completion Record

- ☐ All 20 units completed
- ☐ All labs completed
- ☐ Three capstone projects completed
- ☐ Final exam passed

DEVOPS MASTER COURSE • VOLUME 1 • LINUX

☐ Interview question bank reviewed

☐ Ready for Volume 2 - Bash Shell Scripting and Automation

Learner name: _____

Completion date: _____

Final score: _____

Areas to revise: _____